T.C. SAKARYA UNIVERSITY
FACULTY OF COMPUTER AND INFORMATION SCIENCES
DEPARTMENT OF COMPUTER ENGINEERING

SYSTEM PROGRAMMING COURSE
ASSIGNMENT REPORT

G231210080 — HANSIN EVREN TAŞEL
G221210020 — AYHAN DALKIRAN

2nd Education • Group C
2025–2026 Spring Semester

Q1: Dynamic 2D Matrix with Pointer Arithmetic

1.1 Problem Description

This problem requires writing a C program that allocates an N×N integer matrix dynamically using a single contiguous memory block via malloc, and computes a "special sum" consisting of the main diagonal (i == j) and secondary diagonal (i + j == N−1) elements. If the matrix has an odd dimension, the center element lies on both diagonals and must be counted only once. All matrix element accesses must be performed exclusively through pointer arithmetic; bracket indexing (mat[i][j]) is strictly prohibited.

1.2 Solution Logic

Memory Allocation Strategy

Rather than allocating an array of pointers (one per row), the program allocates a single block of N×N×sizeof(int) bytes. Element at logical position (row, col) is then reached as *(matrix + row*cols + col), which is pure pointer arithmetic equivalent to the flat offset formula. Overflow-safe multiplication is performed before calling malloc:

```c
// Multiplication overflow checks for allocation
size_t element_count = 0;
size_t matrix_size = 0;
if (mul_size_t((size_t)dimension, (size_t)dimension, &element_count) != ERROR_NONE) {
    fprintf(stderr, " fatal error: dimension is too large for matrix to be stored in memory\n");
    return EXIT_FAILURE;
}
else if (mul_size_t(element_count, sizeof(int), &matrix_size) != ERROR_NONE) {
    fprintf(stderr, "fatal error: dimension is too large for matrix to be stored in memory\n");
    return EXIT_FAILURE;
}

// Allocating matrix
int* matrix = (int*)malloc(matrix_size);
if (matrix == NULL) {
    fprintf(stderr, "fatal error: failed to allocate memory for matrix\n");
    return EXIT_FAILURE;
}
```

Multiplication overflow checking logic:

```c
/*
Multiplies two `size_t` values and stores the result in `result_out`.
Returns `error_code_t` indicating success (`ERROR_NONE`) or the type of error that occurred.

Error codes:
`ERROR_MUL_OVERFLOW`: The multiplication of `a` and `b` would overflow the `size_t` type.
*/
error_code_t mul_size_t(size_t a, size_t b, size_t* result_out) {
    if (a > SIZE_MAX / b) return ERROR_MUL_OVERFLOW; // Multiplication overflow

    *result_out = a * b;
    return ERROR_NONE; // Success
}
```

special_sum Algorithm

The function iterates over each row index r from 0 to rows−1. For each row it derives two pointers:

- left_ptr = matrix + r×cols + r → main diagonal element
- right_ptr = matrix + r×cols + (cols−1−r) → secondary diagonal element

Both elements are added to the accumulator. However, when the two pointers are equal (which occurs at the center of an odd-sized matrix, where r == cols−1−r), only left_ptr is added. This single pointer comparison replaces any arithmetic modulo check and keeps the code branch-minimal:

```c
int special_sum(int* matrix, int rows, int cols) {
    int sum = 0;

    for (int r = 0; r < rows; r++) {
        int* left_ptr = matrix + (r * cols) + r; // Primary diagonal element
        int* right_ptr = matrix + (r * cols) + (cols - 1 - r); // Secondary diagonal element

        sum += *left_ptr;
        // If not the center element, add the secondary diagonal element
        if (left_ptr != right_ptr) sum += *right_ptr;
    }

    return sum;
}
```

Command-Line Interface & Robust Input

The program optionally accepts -d <N> to supply the dimension and -f <file> to read matrix elements from a text file instead of the console. When neither flag is present the user is prompted interactively with element-by-row/column labels. Every integer conversion is done through strtol with full errno/range checking, exposing detailed typed error codes via the custom error_code_t enum.

Overflow-safe string to int conversion logic:

```c
// Convert string to long
char* n_str_end = n_str;
errno = 0;
const long n_long = strtol(n_str, &n_str_end, base);

if (errno == ERANGE || n_long < INT_MIN || n_long > INT_MAX) return ERROR_CONVERSION_OUT_OF_RANGE; // Number is out of range
if (n_long == 0 && n_str_end == n_str) return ERROR_CONVERSION_INVALID; // No digits were found
```

1.3 Build & Run

Compilation

The project uses the provided makefile. To build the matrix executable:

```
make matrix
# or to build everything:
make
```

Compiler flags applied: -std=c11 -Wall -Wextra -Werror -pedantic -pedantic-errors

Execution Examples

Interactive mode (dimensions and elements entered at the prompt):

```
./matrix
```

Supplying dimension via flag and elements from a text file:

```
./matrix -d 3 -f matrix.txt
```

The archive includes matrix.txt with the sample 3×3 matrix from the assignment, whose elements are: 1 2 3 / 4 5 6 / 7 8 9.

1.4 Sample Output

(All samples with -f argument only uses matrix.txt provided in archive.)

Run with no -f or -d arguments:

```
~/Desktop/repositories/sau-bsm308-system-programming-assignment 〉main
└ ./matrix
Enter the dimension of the matrix (positive integer): 3
Enter element 1 (row 1, column 1): 1
Enter element 2 (row 1, column 2): 2
Enter element 3 (row 1, column 3): 3
Enter element 4 (row 2, column 1): 4
Enter element 5 (row 2, column 2): 5
Enter element 6 (row 2, column 3): 6
Enter element 7 (row 3, column 1): 7
Enter element 8 (row 3, column 2): 8
Enter element 9 (row 3, column 3): 9
Special sum of the matrix: 25
```

Run with only -d argument provided:

```
~/Desktop/repositories/sau-bsm308-system-programming-assignment 〉main
└ ./matrix -d 3
Enter element 1 (row 1, column 1): 1
Enter element 2 (row 1, column 2): 2
Enter element 3 (row 1, column 3): 3
Enter element 4 (row 2, column 1): 4
Enter element 5 (row 2, column 2): 5
Enter element 6 (row 2, column 3): 6
Enter element 7 (row 3, column 1): 7
Enter element 8 (row 3, column 2): 8
Enter element 9 (row 3, column 3): 9
Special sum of the matrix: 25
```

Run with only -f argument provided:

```
~/Desktop/repositories/sau-bsm308-system-programming-assignment 〉main
└ ./matrix -f matrix.txt
Enter the dimension of the matrix (positive integer): 3
Special sum of the matrix: 25
```

Run with both -f and -d arguments provided:

```
~/Desktop/repositories/sau-bsm308-system-programming-assignment 〉main
└ ./matrix -d 3 -f matrix.txt
Special sum of the matrix: 25
```

Q2: Signal Handling — SIGALRM, SIGINT, SIGSTOP & SIGCONT

2.1 Problem Description

This problem requires writing a C program that demonstrates Unix signal handling by creating a child process with fork(). The child process runs an infinite loop and prints an incrementing counter every second, while handling SIGINT (print message but continue) and SIGCONT (print resume message). The parent process periodically stops and resumes the child using SIGSTOP and SIGCONT via timed SIGALRM cycles, and after approximately 10 seconds sends SIGINT before terminating. Only signal(), alarm(), and kill() system calls may be used.

2.2 Solution Logic

Process Architecture

After fork(), the code branches on the value of child_pid. The child process registers its own signal handlers and enters a tight infinite loop (while(true)), relying entirely on SIGALRM to print the counter. The parent sets up its SIGALRM handler and spins in a busy-wait loop watching the global parent_timer variable.

Parent branch:

```
else { // Parent Process
    signal(SIGALRM, sig_alrm_parent);

    alarm(3);
    while(parent_timer < 10 || !child_running);
    alarm(0);

    fprintf(stdout, "Parent: Sending SIGINT...\n");
    kill(child_pid, SIGINT);

    alarm(1);
    while (parent_timer < 11);
    alarm(0);
}
```

Child branch:

```c
if (child_pid == 0) { // Child Process
    signal(SIGINT, sig_int);
    signal(SIGCONT, sig_cont);
    signal(SIGALRM, sig_alrm_child);
    alarm(1);

    while(true);
}
```

```c
// Child Signal Handler
void sig_int(int signum) {
    (void)signum; // Unused parameter, compiler does not shut up about it
    fprintf(stdout, "Child: SIGINT received but continuing...\n");

    // Override SIG_DFL behaviour
    signal(SIGINT, sig_int);
}

void sig_cont(int signum) {
    (void)signum; // Unused parameter, compiler does not shut up about it
    fprintf(stdout, "Child: Process resumed\n");

    // Override SIG_DFL behaviour
    signal(SIGCONT, sig_cont);
}

void sig_alrm_child(int signum) {
    (void)signum; // Unused parameter, compiler does not shut up about it
    static unsigned int child_timer = 0;
    fprintf(stdout, "Child counter: %u\n", child_timer);
    child_timer++;
    alarm(1);

    // Override SIG_DFL behaviour
    signal(SIGALRM, sig_alrm_child);
}
```

Timer State Machine (Parent)

The parent's alarm handler encodes a simple state machine:

- State RUNNING (child_running == true): send SIGSTOP, set child_running = false, set alarm(2).
- State STOPPED (child_running == false): send SIGCONT, set child_running = true, set alarm(3).
- Termination (parent_timer >= 10): send SIGTERM to child, print confirmation, increment timer to unblock the parent spin-loop.

parent_timer accumulates time (3 s per stop cycle + 2 s per continue cycle), providing an approximate 10-second elapsed measurement without using sleep() or any blocking call:

```c
// Parent Signal Handler
void sig_alrm_parent(int signum) {
    (void)signum; // Unused parameter, compiler does not shut up about it

    if (parent_timer >= 10) { // 10 seconds passed, terminate child
        kill(child_pid, SIGTERM);
        fprintf(stdout, "Parent: Child terminated.\n");
        parent_timer += 1;
    }
    else if (child_running) { // Stop child
        fprintf(stdout, "Parent: Stopping child...\n");
        kill(child_pid, SIGSTOP);
        child_running = false;
        parent_timer += 3;

        alarm(2);
    }
    else { // Continuing child
        fprintf(stdout, "Parent: Continuing child...\n");
        kill(child_pid, SIGCONT);
        child_running = true;
        parent_timer += 2;

        alarm(3);
    }

    signal(SIGALRM, sig_alrm_parent);
}
```

Signal Re-arming

On many POSIX systems the signal disposition resets to SIG_DFL after delivery (System V semantics). To guarantee persistent custom handlers, every handler explicitly calls signal(signum, handler) at its end, re-registering itself. This ensures correct behaviour across both System V and BSD platforms.

SIGSTOP Limitation

SIGSTOP cannot be caught, blocked, or ignored — it is handled exclusively by the kernel. Therefore the child has no SIGSTOP handler; the parent delivers it directly via kill(child_pid, SIGSTOP) and the child is suspended at the OS level.

2.3 Build & Run

Compilation

Build the signal program:

```
make signal
# or to build everything:
make
```

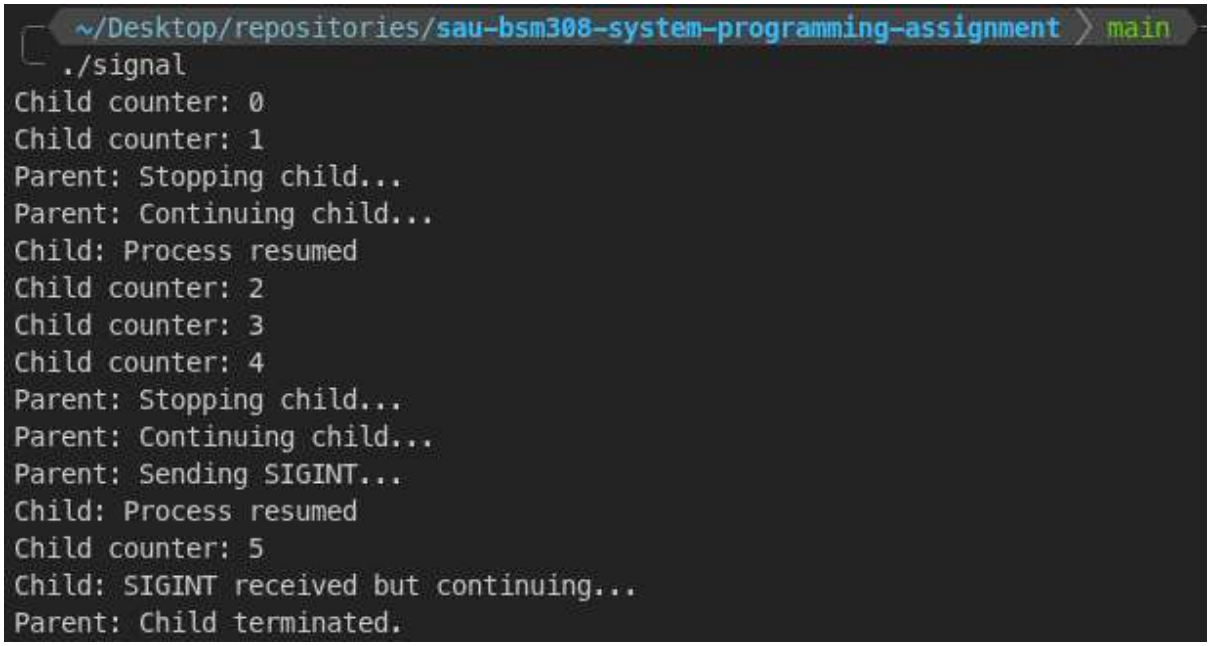
Execution

```
./signal
```

The program should be run on a Linux terminal. It does not accept command-line arguments.

2.5 Sample Output

Expected console output (counter values may differ slightly by timing):



Appendix: Makefile

A single makefile at the root compiles both programs independently or together.

```
# Compiler and flags
CC     = gcc
CFLAGS = -std=c11 -Wall -Wextra -Werror -pedantic -pedantic-errors

.PHONY: all clean

# Default target — build both executables
all: clean matrix signal

clean:
    @rm -f matrix signal

matrix: clean
    @$(CC) $(CFLAGS) matrix.c -o matrix

signal: clean
    @$(CC) $(CFLAGS) signal.c -o signal
```

The -Werror flag turns all warnings into errors, ensuring the code compiles cleanly with zero warnings under strict C11 compliance.